

R Programming: Application-Level Problem Statements for Internal POE

This document contains a curated list of R programming problems designed for B.Tech students. Each problem is structured to test specific core competencies including data structures, manipulation, visualization, and statistical modeling.

1. Sales Performance Analysis

Focus: Vectors & Basic Plotting

Problem: A retail store wants to analyze the weekly sales of two different branches. Create two vectors representing daily sales for 7 days. Calculate the total weekly sales, the average daily sale, and visualize the comparison using a **Bar Chart**.

```
# Data Creation
branch_A <- c(1200, 1500, 1100, 1800, 2000, 2500, 2100)
branch_B <- c(1000, 1300, 1400, 1600, 1900, 2200, 2400)

# Calculations
total_A <- sum(branch_A)
avg_A <- mean(branch_A)

# Visualization
barplot(rbind(branch_A, branch_B), beside = TRUE,
        col = c("blue", "green"),
        names.arg = c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat",
"Sun"),
        main = "Weekly Sales Comparison", xlab = "Day", ylab =
"Sales ($)")
legend("topleft", legend = c("Branch A", "Branch B"), fill =
c("blue", "green"))
```

2. Multi-Dimensional Sensor Data

Focus: Arrays

Problem: A weather station records temperature and humidity across two different zones over four time intervals. Store this data in a **3D Array**. Calculate the mean temperature for each zone across all time intervals.

```
# Creating a 2x4x2 array (2 Zones, 4 Time slots, 2 Metrics: Temp &
Humidity)
```

```

data_values <- c(22, 24, 21, 25, 30, 31, 29, 32, 60, 65, 70, 75, 55,
50, 58, 62)
weather_array <- array(data_values, dim = c(2, 4, 2),
                        dimnames = list(c("Zone1", "Zone2"),
                                         c("T1", "T2", "T3", "T4"),
                                         c("Temp", "Humidity")))

# Mean Temperature for Zone 1
mean_zone1_temp <- mean(weather_array["Zone1", , "Temp"])
print(weather_array)
print(paste("Mean Temp Zone 1:", mean_zone1_temp))

```

3. Student Grade Frequency

Focus: R Tables & Pie Charts

Problem: Given a list of grades for 50 students, generate a **Frequency Table** to show how many students received each grade (A, B, C, D, F). Represent the distribution using a **Pie Chart**.

```

# Generating random grade data
grades <- sample(c("A", "B", "C", "D", "F"), 50, replace = TRUE)

# Frequency Table
grade_table <- table(grades)
print(grade_table)

# Pie Chart
pie(grade_table, main = "Grade Distribution", col = rainbow(5))

```

4. Employee Database Management

Focus: Data Manipulation & Shaping

Problem: Create a data frame containing Employee ID, Name, and Department. Filter the data to show only employees in the "IT" department and **Sort** the resulting list by Name. Use basic R to transform data from wide to long format.

```

# Data Frame
emp_data <- data.frame(ID = 1:4,
                        Name = c("Alice", "Bob", "Charlie", "David"),
                        Dept = c("IT", "HR", "IT", "Finance"),
                        Salary = c(50000, 45000, 55000, 60000))

```

```
# Filtering and Sorting
it_employees <- emp_data[emp_data$Dept == "IT", ]
sorted_it <- it_employees[order(it_employees$Name), ]

# Shaping: Wide to Long using stack/reshape
long_data <- reshape(emp_data, direction="long", varying=list(4),
v.names="Value", times="Salary", idvar="ID")
print(sorted_it)
print(head(long_data))
```

5. Healthcare Record Merging

Focus: Merging & Data Structures

Problem: You have two datasets: one with Patient ID and Personal Info, and another with Patient ID and Medical Diagnosis. **Merge** these datasets into a single data frame and identify any missing records.

```
# Datasets
personal <- data.frame(PID = c(101, 102, 103), Name = c("John",
"Jane", "Doe"))
medical <- data.frame(PID = c(101, 103, 104), Diagnosis = c("Flu",
"Cold", "Fever"))

# Inner Merge
complete_records <- merge(personal, medical, by = "PID")

# Full Merge (includes NAs for missing info)
all_records <- merge(personal, medical, by = "PID", all = TRUE)
print(all_records)
```

6. Population Growth Distribution

Focus: Histograms

Problem: Simulate a dataset of 100 city populations. Create a **Histogram** to visualize the distribution frequency of population sizes and add a density line to the plot.

```
# Data Stream Simulation
set.seed(123)
populations <- rnorm(100, mean = 500000, sd = 100000)

# Histogram
hist(populations, breaks = 10, col = "orange", border = "black",
```

```
    main = "City Population Distribution", xlab = "Population  
Count")
```

7. Predicting House Prices

Focus: Linear Regression

Problem: Given a dataset of House Size (sq ft) and Price, build a **Linear Regression** model to predict the price of a house. Plot the data points and the regression line.

```
# Data
sqft <- c(1500, 1800, 2400, 3000, 3500)
price <- c(300000, 350000, 450000, 500000, 600000)

# Model Building
model <- lm(price ~ sqft)
summary(model)

# Plotting
plot(sqft, price, main = "Price vs Size", pch = 19, col = "blue")
abline(model, col = "red", lwd = 2)
```

8. Real-time Log Monitoring

Focus: Data Stream Simulation

Problem: Simulate a continuous **Data Stream** of CPU usage percentages. Write a loop that processes input, calculates a running average every 10 steps, and stops if the usage exceeds 95%.

```
# Simulating a stream loop
set.seed(42)
stream_buffer <- c()

for(i in 1:100) {
  current_usage <- runif(1, 40, 100) # Simulating input
  stream_buffer <- c(stream_buffer, current_usage)

  if(i %% 10 == 0) {
    cat("Batch", i/10, "- Average Usage:", mean(stream_buffer),
        "\n")
  }

  if(current_usage > 95) {
```

```
        print(paste("Critical Alert! Usage reached:",  
round(current_usage, 2)))  
        break  
    }  
}
```